

Gradle Project

Introduction to Gradle

Gradle is an open-source build automation tool used mainly for software development projects. In DevOps, Gradle helps automate tasks such as compiling code, testing, packaging, dependency management, and deployment. It is widely used for Java, Android, Kotlin, and many other application types.

Gradle combines the best features of tools like Ant and Maven while providing greater flexibility and faster performance. It uses a Groovy or Kotlin-based Domain Specific Language (DSL) for writing build scripts.

In a DevOps environment, automation is very important for Continuous Integration (CI) and Continuous Deployment (CD). Gradle plays a major role by automating the software build process.

Key Features of Gradle

1. Build Automation
 - Automatically compiles source code, runs tests, and creates executable files.
2. Dependency Management
 - Downloads and manages required libraries from repositories automatically.
3. Fast Performance
 - Uses incremental builds and build caching to reduce build time.
4. Flexibility
 - Supports multiple programming languages and project structures.
5. Integration with DevOps Tools
 - Easily integrates with tools like:
 - Jenkins
 - Docker
 - Kubernetes
 - Git
6. CI/CD Support
 - Helps automate build and deployment pipelines in DevOps workflows.

Working of Gradle in DevOps

Typical workflow:

1. Developer writes code.
 2. Code is pushed to Git repository.
 3. Jenkins triggers Gradle build automatically.
 4. Gradle compiles code and runs tests.
 5. Application package is generated.
 6. Deployment tools deploy the application to servers or containers.
-

Advantages of Gradle

- Faster builds
- Reduced manual work
- Better project management
- Easy customization
- Supports continuous delivery

Conclusion

Gradle is an important DevOps tool used for automating software builds and managing dependencies. It improves development speed, supports CI/CD pipelines, and helps teams deliver applications efficiently and reliably.

Steps to Execute the project:

Step 1:

Create a new Gradle project

To create a new Gradle project using the command line:

\$gradle init --type java-application

```
vaishnavi@vaishnavi-virtualbox:~/devops21$ mkdir MyGradleApp
vaishnavi@vaishnavi-virtualbox:~/devops21$ cd MyGradleApp/
vaishnavi@vaishnavi-virtualbox:~/devops21/MyGradleApp$ gradle init --type java-application
Starting a Gradle Daemon (subsequent builds will be faster)

BUILD SUCCESSFUL in 5s
2 actionable tasks: 2 executed
vaishnavi@vaishnavi-virtualbox:~/devops21/MyGradleApp$
```

This command creates a new Java application project with a sample **build.gradle** file.

Step 2:

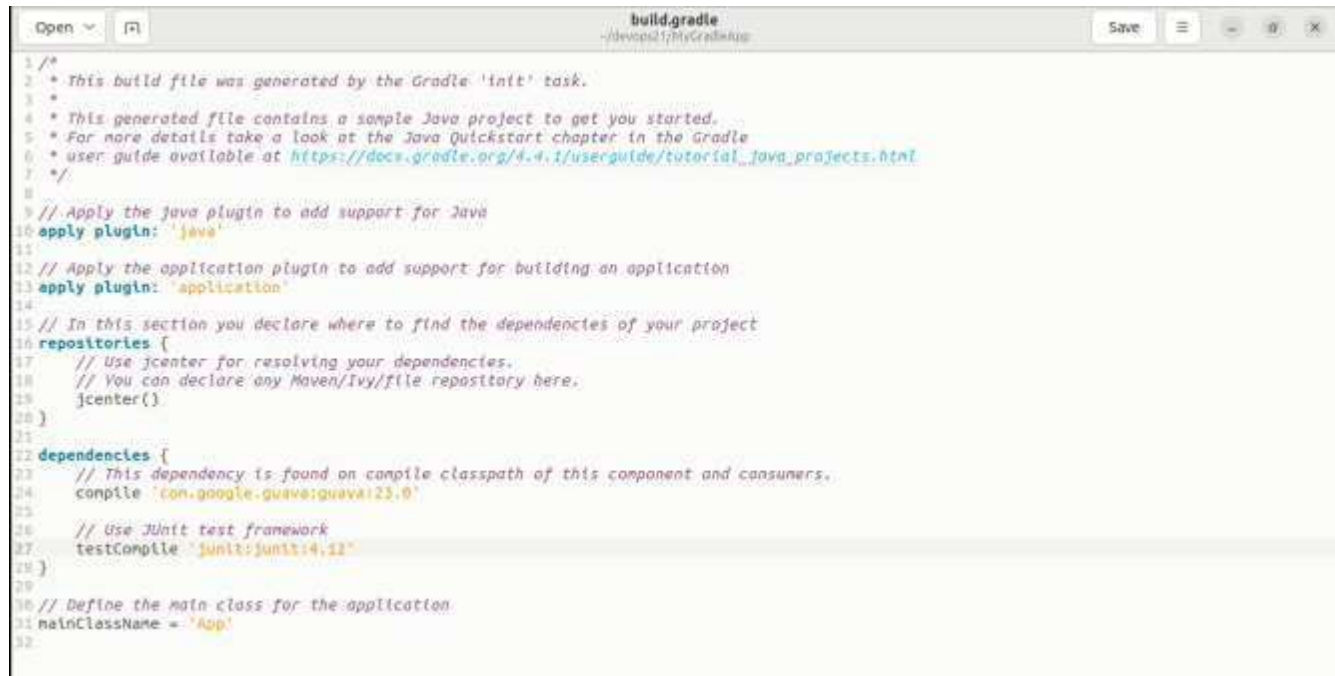
Execute \$tree command to view the project structure

```
vaishnavi@vaishnavi-virtualbox:~/devops21/MyGradleApp$ tree
.
├── build.gradle
├── gradle
│   └── wrapper
│       ├── gradle-wrapper.jar
│       └── gradle-wrapper.properties
├── gradlew
├── gradlew.bat
├── settings.gradle
└── src
    ├── main
    │   └── java
    │       └── App.java
    └── test
        └── java
            └── AppTest.java

7 directories, 8 files
```

Step 3:

Edit the build.gradle file to add necessary plugins



```
1 /*
2  * This build file was generated by the Gradle 'init' task.
3  *
4  * This generated file contains a sample Java project to get you started.
5  * For more details take a look at the Java Quickstart chapter in the Gradle
6  * user guide available at https://docs.gradle.org/4.4.1/userguide/tutorial\_java\_projects.html
7  */
8
9 // Apply the java plugin to add support for Java
10 apply plugin: 'java'
11
12 // Apply the application plugin to add support for building an application
13 apply plugin: 'application'
14
15 // In this section you declare where to find the dependencies of your project
16 repositories {
17     // Use jcenter for resolving your dependencies.
18     // You can declare any Maven/Ivy/file repository here.
19     jcenter()
20 }
21
22 dependencies {
23     // This dependency is found on compile classpath of this component and consumers.
24     compile 'com.google.guava:guava:23.0'
25
26     // Use JUnit test framework
27     testCompile 'junit:junit:4.12'
28 }
29
30 // Define the main class for the application
31 mainClassName = 'App'
32
```

- **build.gradle** is a configuration file used by Gradle to build and manage a project
- **Plugins section**
 - java → enables Java compilation
 - application → allows running the project as an application
- **Repositories section**
 - Defines where dependencies are downloaded from
 - jcenter() → a repository for libraries
- **Dependencies section**
 - guava → provides utility functions for Java
 - junit → used for testing the application
- **Main Class**
 - mainClassName = 'App' → specifies the entry point of the program
- **Overall purpose**
 - Manages dependencies
 - Builds the project
 - Runs the application

Step 4:

Execute the given command to run the gradle project

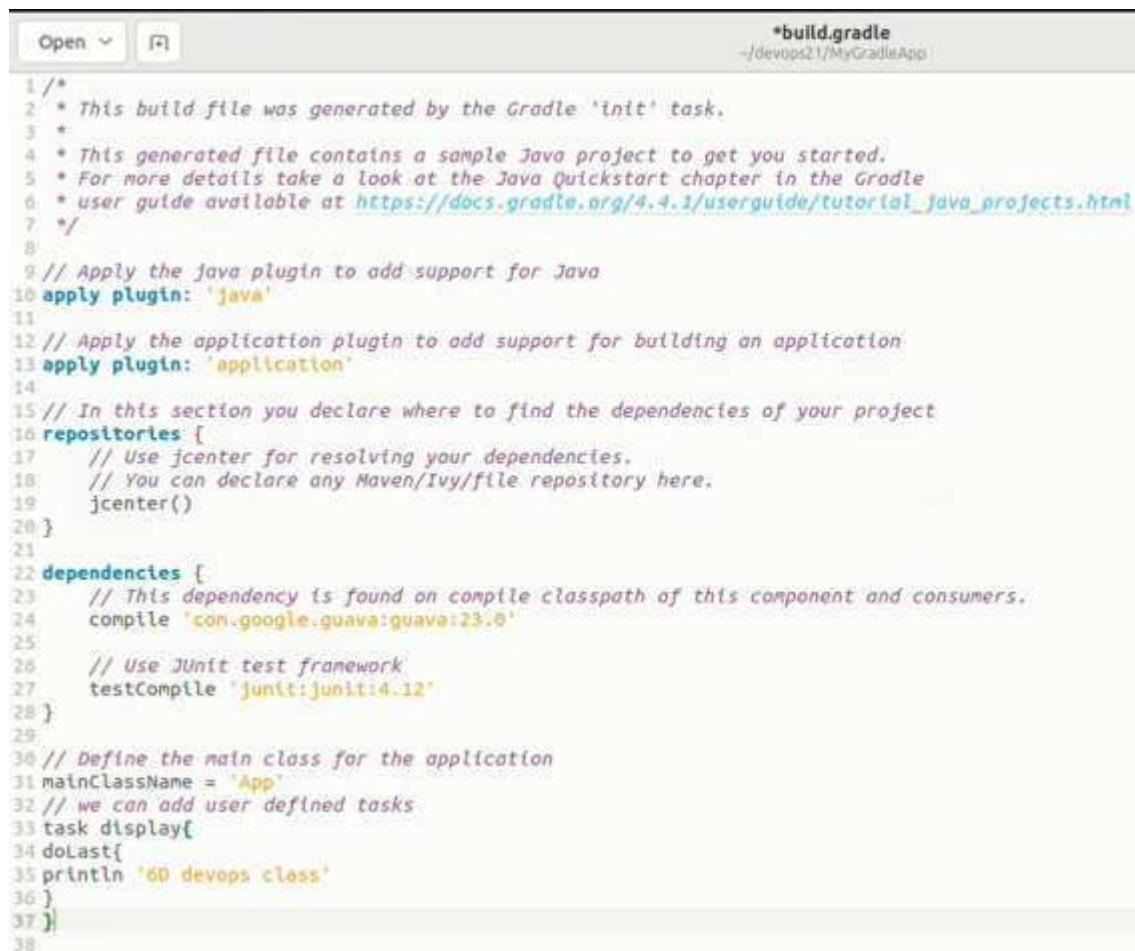
\$gradle run

```
vaishnavi@vaishnavi-virtualbox:~/devops21/MyGradleApp$ gradle run
> Task :run
Hello world.

BUILD SUCCESSFUL in 1s
2 actionable tasks: 1 executed, 1 up-to-date
vaishnavi@vaishnavi-virtualbox:~/devops21/MyGradleApp$
```

Step 5:

Add the user defined tasks to build.gradle at the end

The screenshot shows an IDE window titled '*build.gradle' with the file path '~/devops21/MyGradleApp'. The file contains Gradle build script code. At the bottom, a new task named 'display' is defined, which prints the message '60 devops class'.

```
1 /*
2  * This build file was generated by the Gradle 'init' task.
3  *
4  * This generated file contains a sample Java project to get you started.
5  * For more details take a look at the Java Quickstart chapter in the Gradle
6  * user guide available at https://docs.gradle.org/4.4.1/userguide/tutorial\_java\_projects.html
7  */
8
9 // Apply the java plugin to add support for Java
10 apply plugin: 'java'
11
12 // Apply the application plugin to add support for building an application
13 apply plugin: 'application'
14
15 // In this section you declare where to find the dependencies of your project
16 repositories {
17     // Use jcenter for resolving your dependencies.
18     // You can declare any Maven/Ivy/file repository here.
19     jcenter()
20 }
21
22 dependencies {
23     // This dependency is found on compile classpath of this component and consumers.
24     compile 'com.google.guava:guava:23.0'
25
26     // Use JUnit test framework
27     testCompile 'junit:junit:4.12'
28 }
29
30 // Define the main class for the application
31 mainClassName = 'App'
32 // we can add user defined tasks
33 task display{
34     doLast{
35         println '60 devops class'
36     }
37 }
38
```

Step 6:

To run user defined task execute the commands

```
vaishnavi@vaishnavi-virtualbox:~/devops21/MyGradleApp$ gedit build.gradle
vaishnavi@vaishnavi-virtualbox:~/devops21/MyGradleApp$ gradle display
```

```
> Task :display
6D devops class
```

```
BUILD SUCCESSFUL in 1s
1 actionable task: 1 executed
vaishnavi@vaishnavi-virtualbox:~/devops21/MyGradleApp$ █
```

Pushing The Project into Git:

Execute the commands:

```
$ git config --global user.name "vaishnavim1121"
```

```
$ git config --global user.email "vaishnavim1121@gmail.com"
```

```
$git init
```

```
$git add .
```

```
$git commit -m "initial commit"
```

```
$ git remote add origin https://github.com/vaishnavim1121/MyMavenApp.git
```

```
$ git remote set-url origin https://*PAT*@github.com/vaishnavim1121/MyMavenApp
```

```
$ git push -u origin master
```

```
AppTest.class

30 directories, 18 files
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to one of all
hint: lists of your new remotes, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:   git branch -m <new>
Initialized empty Git repository in /home/vaishnavi/devops21/MyMavenToGradle/.git/
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle$ git add .
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle$ git commit -m "maven to gradle"
[master (root-commit) 255ca20] maven to gradle
 27 files changed, 473 insertions(+)
 create mode 100644 .gradle/4.4.1/fileChanges/last-build.bin
 create mode 100644 .gradle/4.4.1/fileHashes/fileHashes.bin
 create mode 100644 .gradle/4.4.1/fileHashes/fileHashes.lock
 create mode 100644 .gradle/4.4.1/taskHistory/taskHistory.bin
 create mode 100644 .gradle/4.4.1/taskHistory/taskHistory.lock
 create mode 100644 .gradle/buildOutputCleanup/buildOutputCleanup.lock
 create mode 100644 .gradle/buildOutputCleanup/cache.properties
 create mode 100644 .gradle/buildOutputCleanup/outputFiles.bin
 create mode 100644 build.gradle
 create mode 100644 gradle/wrapper/gradle-wrapper.jar
 create mode 100644 gradle/wrapper/gradle-wrapper.properties
 create mode 100755 gradlew
 create mode 100644 gradlew.bat
 create mode 100644 pom.xml
 create mode 100644 settings.gradle
 create mode 100644 src/main/java/com/example/App.java
 create mode 100644 src/test/java/com/example/AppTest.java
 create mode 100644 target/MavenApp-1.0-SNAPSHOT.jar
 create mode 100644 target/classes/com/example/App.class
 create mode 100644 target/maven-archiver/pom.properties
 create mode 100644 target/maven-status/maven-compiler-plugin/compile/default-compile/createdFiles.lst
 create mode 100644 target/maven-status/maven-compiler-plugin/compile/default-compile/inputFiles.lst
 create mode 100644 target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/createdFiles.lst
 create mode 100644 target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/inputFiles.lst
 create mode 100644 target/surefire-reports/TEST-com.example.AppTest.txt
 create mode 100644 target/surefire-reports/com.example.AppTest.txt
 create mode 100644 target/test-classes/com/example/AppTest.class
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle$ git remote add origin https://github.com/vaishnavim1121/MyMavenToGradle.git
git: 'remote' is not a git command. See 'git --help'.
```

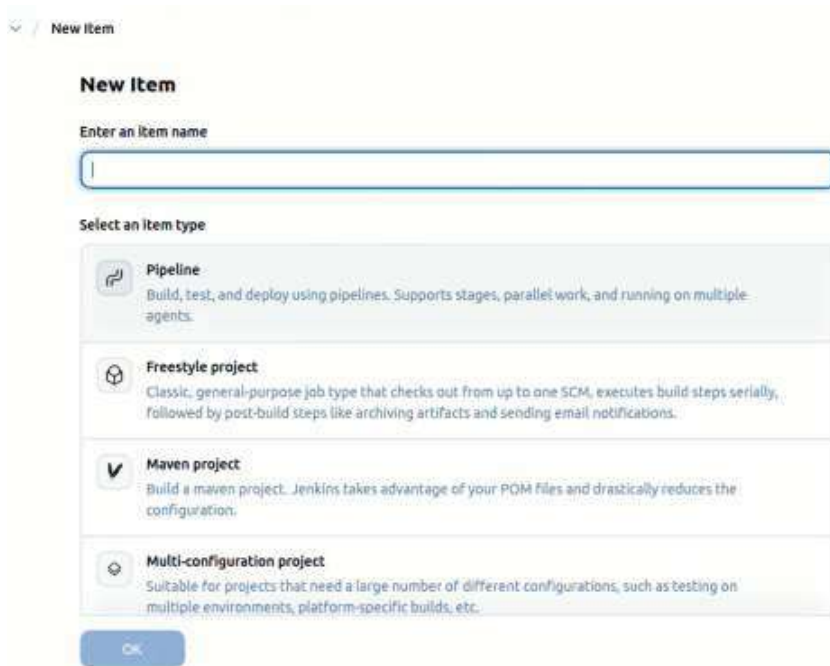
Second Phase

Depicting Jenkins Pipeline

Follow the given steps to deploy the project on Jenkins.

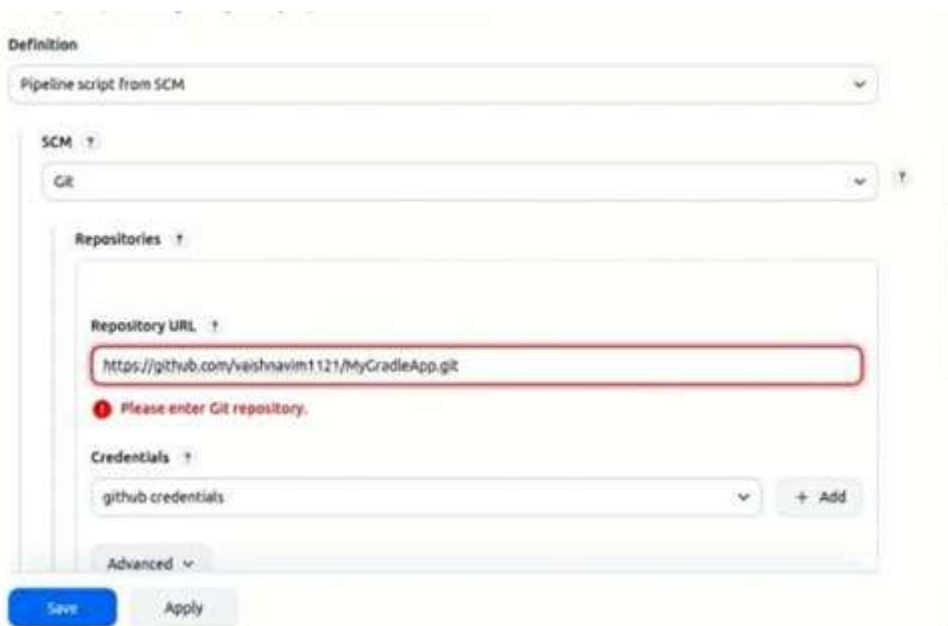
create a Jenkins pipeline job that will use the Jenkinsfile from your project repository

go to the Jenkins home page and click on “New Item” to create a job, enter the item name select Pipeline as the project type.



The image shows the 'New Item' form in Jenkins. At the top, there is a 'New Item' link. Below it, the title 'New Item' is displayed. A text input field for 'Enter an item name' is present. Under the 'Select an item type' section, four options are listed: 'Pipeline' (with a description: 'Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.'), 'Freestyle project' (description: 'Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.'), 'Maven project' (description: 'Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.'), and 'Multi-configuration project' (description: 'Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.'). The 'Pipeline' option is selected, indicated by a checkmark. At the bottom, there is an 'OK' button.

This option allows you to build, test, and deploy applications using a Jenkinsfile. After selecting the pipeline type, click “OK” to proceed to the job configuration page



The image shows the 'Pipeline Configuration' form in Jenkins. The 'Definition' section has a dropdown menu set to 'Pipeline script from SCM'. The 'SCM' section has a dropdown menu set to 'Git'. The 'Repositories' section is expanded, showing a 'Repository URL' input field with the value 'https://github.com/veishnavim1121/MyGradleApp.git'. Below the URL, there is a red error message: 'Please enter Git repository.' The 'Credentials' section has a dropdown menu set to 'github credentials' and an '+ Add' button. At the bottom, there are 'Save' and 'Apply' buttons.

Now we need to add a Jenkinsfile into our project which will consist of the operations that will occur when building the project in Jenkins

It includes step wise instructions guiding at each stage with right commands

```

1  pipeline {
2      agent any // Use any available agent
3
4      tools {
5          gradle 'Gradle' // Ensure this matches the name configured in Jenkins
6          jdk 'JDK'
7      }
8      stages {
9          stage('Checkout') {
10             steps {
11                 git branch: 'master', url: 'https://github.com/vaishnavim1121/MyGradleApp.git'
12             }
13         }
14
15         stage('Build') {
16             steps {
17                 sh 'gradle build' // Run Gradle build
18             }
19         }
20
21         stage('Test') {
22             steps {
23                 sh 'gradle test' // Run unit tests
24             }
25         }
26
27
28
29
30         stage('Run Application') {
31             steps {
32                 // Start the JAR application
33                 sh 'gradle run'

```

Now try building the project with Jenkins

